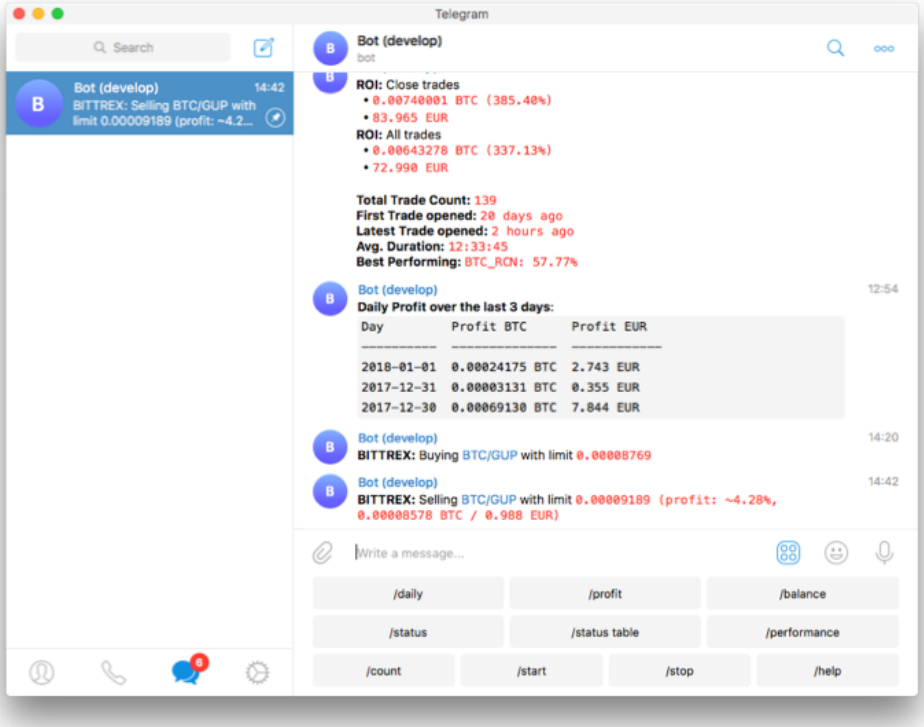
# Platform Documentation

Freqtrade · Binance · Python RSI Strategy

Version 1.0 · June 2026

■ Automated Trading | ■ Live + Backtesting | ■ Paper / Real Money | ■ Docker Powered




*FreqUI — Live trading dashboard*

Prepared for: Aris Magklaras | Exchange: Binance | Mode: Dry-run → Live

# Table of Contents

# 1 Platform Overview

This platform runs **Freqtrade**, a battle-tested open-source algorithmic crypto trading bot, connected to **Binance** using a Python-based RSI strategy. It is containerised with **Docker**, so the entire stack runs with a single command and works identically on any machine.

| Component | Technology | Purpose |
|---|---|---|
| Bot engine | Freqtrade (stable) | Executes strategies, manages orders |
| Exchange | Binance (spot) | Market data & order execution |
| Strategy | Python — RSIStrategy | Buy/sell signal logic |
| Infrastructure | Docker + Compose | Isolated, reproducible environment |
| Web UI | FreqUI (port 8080) | Monitor trades & performance |
| Currency | USDT | All trades quoted in USDT |
| Mode | Dry-run (paper) → Live | Safe testing before real money |

■ The bot is currently running in **dry-run mode**: it makes real trades in simulation using live Binance prices, but no real money is at risk. Your starting paper balance is **1000 USDT**.

# 2 Project File Structure

All files live in **~/GitHub/AlgoTrading/**:

```
AlgoTrading/
■■■ docker-compose.yml          ← starts/stops the bot
■■■ .gitignore                  ← keeps secrets out of git
■■■ README.md
■■■ user_data/
    ■■■ config.json             ← ALL bot settings
    ■■■ strategies/
    ■   ■■■ RSIStrategy.py      ← YOUR strategy code
    ■■■ logs/                   ← runtime logs
    ■■■ data/                   ← downloaded OHLCV data
    ■■■ backtest_results/       ← backtest reports
```

| File | What to edit there |
|---|---|
| config.json | Pairs, stake size, dry_run on/off, API keys, stop-loss |
| RSIStrategy.py | Buy/sell signal logic — the actual algorithm |
| docker-compose.yml | Port, strategy name, log path |

# 3 Starting & Stopping the Bot

## Start the bot

Open Terminal and navigate to the project folder:

```
cd ~/GitHub/AlgoTrading
docker compose up -d
```

The **-d** flag runs it in the background. The bot will start trading (dry-run) automatically.

## Check status

```
docker ps
```

You should see a container named **freqtrade** with status *Up*.

## View live logs

```
docker logs freqtrade -f
```

Press Ctrl+C to stop watching logs (the bot keeps running).

## Restart after config changes

```
docker compose restart
```

## Stop the bot completely

```
docker compose down
```

> Always restart after editing **config.json** or **RSIStrategy.py** — the bot reads these files only at startup.

# 4 The FreqUI Web Interface

Open **http://localhost:8080** in your browser while the bot is running.

**Login credentials**

```
Username:  freqtrade

Password:  change-this-password
```

## Dashboard

The main screen shows your **current balance**, **profit/loss summary**, number of open and closed trades, and real-time price charts for each pair being watched.

## Trades Tab

Lists every trade the bot has made or currently holds. For each trade you can see: pair, entry price, current price, profit %, duration, and the reason it was opened/closed.

| Column | Meaning |
|---|---|
| Pair | The crypto pair traded, e.g. BTC/USDT |
| Open Rate | Price at which the bot bought |
| Current Rate | Current live price |
| Profit % | Unrealised gain or loss |
| Open Date | When the position was entered |
| Close Reason | ROI / Stop Loss / Sell Signal |

## Logs Tab

Shows the bot's internal log in real-time. Look for lines starting with **INFO** (normal operation), **WARNING** (something unexpected but recoverable), or **ERROR** (needs attention).

# 5 Understanding the RSI Strategy

The strategy file is at **user_data/strategies/RSIStrategy.py**. It combines two classical technical indicators: **RSI** (momentum) and **EMA** (trend direction).

## How RSI Works

The **Relative Strength Index (RSI)** is a number between 0 and 100 that measures how fast and how much a price has recently moved:

- **RSI < 30** → the asset is **oversold**: it has dropped quickly and may bounce back up
- **RSI > 70** → the asset is **overbought**: it has risen quickly and may pull back
- **RSI = 50** → neutral momentum

## Entry Signal (Buy)

The bot opens a trade when **both** conditions are true:

- RSI falls below **30** (oversold — potential bounce)
- The 20-period EMA is above the 50-period EMA (confirming an **uptrend**)

> The EMA filter prevents buying into a falling market. RSI alone can give false signals in a strong downtrend — the EMA cross adds a second layer of confirmation.

## Exit Signal (Sell)

The bot closes a trade when **any** of these conditions are met:

- RSI rises above **70** (overbought — take profit signal)
- Profit reaches **+4%** at any time
- Profit reaches **+2%** after 30 minutes
- Profit reaches **+1%** after 60 minutes
- Trade breaks even (+0%) after 2 hours

## Stop Loss

If the trade drops **-5%** below the entry price, the bot closes it immediately with a market order to limit losses.

## Key Parameters Summary

| Parameter | Value | Location in code |
|---|---|---|
| RSI period | 14 candles | rsi_period.default |
| Buy RSI threshold | < 30 | buy_rsi.default |
| Sell RSI threshold | > 70 | sell_rsi.default |

| | | |
|---|---|---|
| Stop loss | -5% | stoploss = -0.05 |
| Timeframe | 5 minutes | timeframe = '5m' |
| Max open trades | 3 | config.json |
| Stake per trade | 100 USDT | config.json |

# 6 Modifying the Strategy

Open **user_data/strategies/RSIStrategy.py** in any text editor (VS Code recommended). After saving changes, restart the bot:

```
docker compose restart
```

## Change the trading pairs

Edit **user_data/config.json**, section *pair_whitelist*:

```
"pair_whitelist": [
  "BTC/USDT",
  "ETH/USDT",
  "SOL/USDT"
]
```

## Change RSI thresholds

In RSIStrategy.py, adjust the default values:

```
buy_rsi  = IntParameter(low=20, high=40, default=30, ...)  # lower = more selective
sell_rsi = IntParameter(low=60, high=80, default=70, ...)  # higher = more selective
```

## Change stop loss

In RSIStrategy.py:

```
stoploss = -0.05  # -5%. Change to -0.03 for tighter, -0.10 for looser
```

## Change stake amount per trade

In config.json:

```
"stake_amount": 100  # USDT per trade
```

## Change timeframe

The bot currently checks for signals every **5 minutes**. To trade less frequently (fewer but potentially stronger signals):

```
timeframe = "15m"  # or "1h" for hourly signals
```

> Always backtest after any strategy change before going live. A parameter that looks good in theory can perform poorly in real markets.

# 7 Backtesting

Backtesting simulates your strategy against historical market data so you can see how it *would have* performed — before risking any money.

## Step 1 — Download historical data

```
docker compose run --rm freqtrade download-data \
  --config /freqtrade/user_data/config.json \
  --days 90 \
  --timeframe 5m
```

## Step 2 — Run the backtest

```
docker compose run --rm freqtrade backtesting \
  --config /freqtrade/user_data/config.json \
  --strategy RSIStrategy \
  --timerange 20240101-20241231
```

## Reading the results

| Metric | What it means |
|---|---|
| Total profit % | Overall return for the period |
| Win rate | % of trades that closed in profit |
| Max drawdown | Biggest peak-to-trough loss — key risk metric |
| Avg trade duration | How long positions are typically held |
| Best/Worst trade | Range of single-trade outcomes |
| Sharpe ratio | Return relative to risk (higher is better, > 1 is good) |

Backtesting shows past performance, which does not guarantee future results. Always validate with at least 2–4 weeks of dry-run before going live.

# 8 Going Live with Real Money

> Only proceed after the strategy has run profitably in dry-run for at least 2 weeks AND you have run a successful backtest. Start with the smallest amount you are comfortable losing entirely.

## Step 1 — Create Binance API keys

- Log into Binance → click your profile icon → **API Management**
- Click **Create API** → choose **System generated**
- Label it *freqtrade*
- Enable: **Enable Reading** ✓ and **Enable Spot & Margin Trading** ✓
- **NEVER enable withdrawals** — this keeps your funds safe even if keys leak
- Copy the **API Key** and **Secret Key** — the secret is shown only once
- Restrict the key to your home IP address for extra security

## Step 2 — Add keys to config.json

```
"exchange": {
  "name": "binance",
  "key": "paste-your-api-key-here",
  "secret": "paste-your-secret-here",
  ...
}
```

## Step 3 — Disable dry-run

```
"dry_run": false
```

## Step 4 — Set a safe stake amount

For your first live run, use a small amount — e.g. 20–30 USDT per trade:

```
"stake_amount": 20
```

## Step 5 — Restart

```
docker compose restart
```

> The bot will now place real orders on Binance. Watch the Trades tab closely for the first hour. You can stop it at any time with **docker compose down**.

# 9 Troubleshooting

| | |
|---|---|
| **Problem: Bot container exits immediately** | **Solution:** Run **docker logs freqtrade** and look for CRITICAL or ERROR lines. The most common causes are invalid config.json syntax or a missing required field. |
| **Problem: Web UI doesn't load at localhost:8080** | **Solution:** Confirm the container is running: **docker ps**. If not listed, run **docker compose up -d** again. |
| **Problem: 'No data found' during backtest** | **Solution:** Run the download-data command first (Section 7, Step 1). |
| **Problem: Bot opens no trades after hours** | **Solution:** The RSI + EMA conditions may not be met in the current market. This is normal — the bot only trades when its signal fires. Check the Logs tab for 'No buy tag found' messages. |
| **Problem: Exchange error / rate limit** | **Solution:** Binance has API rate limits. Freqtrade respects them automatically. If you see repeated errors, check your Binance API key permissions. |
| **Problem: Config changes not taking effect** | **Solution:** Remember to restart: **docker compose restart**. |

For more help: **https://www.freqtrade.io/en/stable/** | Community: **https://discord.gg/p7nuUNVfP7**